



Proton: Towards Multi-level, Adaptive Profiling for Triton

Keren Zhou^{*¶}, Tianle Zhong[†], Hao Wu^{*}, Jihyeong Lee^{*}, Yue Guan[‡],
Yufei Ding[‡], Corbin Robeck[§], Yuanwei Fang[§], Jeff Niu[¶] and Philippe Tillet[¶]

^{*}Department of Computer Science, George Mason University, Fairfax, VA, USA
{kzhou6, hwu27, jlee436}@gmu.edu

[†]University of Virginia, Charlottesville, VA, USA
fad3ew@virginia.edu

[‡]University of California, San Diego, La Jolla, CA, USA

{y9guan, yufeidng}@ucsd.edu

[§]Meta, Menlo Park, CA, USA

{robeck, fywkevin}@meta.com

[¶]OpenAI, San Francisco, CA, USA

{jeffniu, phil}@openai.com

Abstract—Domain-Specific languages (DSLs) such as Triton enable developers to write high-performance GPU kernels in a Python-friendly manner; however, profiling these kernels with existing tools often incurs runtime and storage overhead while failing to deliver actionable insights for both kernel authors and framework developers. We present PROTON, a multi-level, adaptive profiler tailored for the Triton programming language and compiler. PROTON provides frontend APIs to selectively profile relevant regions, aggregate results, capture custom metrics not available through hardware counters, and query profiles using a SQL-like language. PROTON’s backend design unifies vendor profiling APIs with instrumentation-based profiling, ensuring portability and extensibility. Using PROTON, users are able to query custom and hardware metrics across the relevant levels of abstraction—full end-to-end model execution, isolated neural network layers, language-specific Triton operators, and compiler intermediate representations. We demonstrate the tool’s effectiveness through case studies on production-grade kernel development, continuous integration, multi-GPU analysis, language model inference, and intra-kernel profiling. Our evaluations on end-to-end workloads, as well as standalone Triton kernels, demonstrate that PROTON imposes lower runtime overhead and delivers significant reductions in profile sizes relative to existing framework and vendor profilers while being fully open source.

Index Terms—Performance Profiling, Program Analysis, GPUs, Artificial Intelligence.

I. INTRODUCTION

GPUs have fueled the widespread adoption of modern AI applications, including large language models (LLMs) [1], [2], image generation tools [3], [4], speech recognition systems [5], robotic platforms [6], [7], and real-time recommendation systems [8]. However, the growing complexity of GPU architectures, driven by innovations such as Tensor Cores [9], [10] and Tensor Memory Accelerators [11], [12], has made GPU programming increasingly challenging. Developing GPU kernels with these specialized units using low-level programming languages, and often even hand-written GPU assembly (e.g., CUDA+PTX [13]), requires not only careful management of complex architectural details but also restricts

the flexibility to rapidly prototype algorithm variants. As a result, domain-specific AI programming models, including Triton [14], CuTile [15], TileLang [16], ThunderKittens [17], and MosaicGPU [18], have emerged to address the challenges, providing Python interfaces, tensor-centric semantics, and simplified parallelism.

Among these programming models, Triton has seen the most substantial adoption due to its highly optimized compiler backends for AMD and NVIDIA GPUs, user-friendly syntax, open source software approach, and seamless integration with popular AI frameworks such as PyTorch [19]. Triton simplifies GPU programming while delivering performance comparable to manually optimized CUDA code in many cases, such as FlashAttention [20] and Mixture-of-Experts [21], making it especially valuable for performance-critical AI models. Yet, quickly understanding and optimizing Triton-written kernels remains a challenge. This has become increasingly important as the pace of kernel iteration within modern AI frameworks very often causes the time taken to optimize a kernel to exceed its lifetime in the framework. Existing studies [22], [23] report that developing efficient Triton kernels often requires multiple rounds of tuning and may remain elusive for researchers, even when an optimal kernel written in Triton theoretically exists.

The challenges of analyzing and optimizing Triton kernels arise at different levels of abstraction, depending on the specific type of user and their priorities (Table I). For AI researchers who are primarily interested in prototyping new algorithms (*level 1*), the key requirement is the ability to quickly identify high-impact kernels that change directly as they refine their designs. Performance engineers, on the other hand (*level 2*), are more concerned with developing portable, high-performance kernels across a range of inputs. For them, it is essential to understand how variations in launch configurations translate into differences in efficiency, which requires visibility into both custom metrics and low-level hardware counters. Finally, there are compiler engineers

TABLE I: Comparison of profilers

Level	Feature	Vendor	TorchProf	PROTON
1	Kernel annotation (e.g., <code>kernel_{shape}_{constrs}</code>)	Limited	Limited	✓
1	Python API (e.g., <code>start/skip/finalize</code>)	×	✓	✓
2	Custom / derived metrics	×	×	✓
2	Hardware counters	✓	×	✓
3	IR-level instrumentation	×	×	✓
—	Runtime and storage overhead	High	High	Low

working on GPU backend optimizations that operate at the lowest level of abstraction (*level 3*). Their focus is on how intermediate representations (IRs) impact performance, and they rely on fine-grained instrumentation to reveal effects such as how asynchronous instructions overlap and the stalls related to instruction scheduling.

Profiling remains an essential tool for analyzing inefficiencies and providing insights for optimizing Triton kernels. While existing profilers such as NVIDIA’s Nsight Systems [24] (*nsys*), Nsight Compute [25] (*ncu*), and PyTorch’s Profiler (TorchProf) offer valuable insights, they exhibit notable shortcomings when applied to AI programming models like Triton, as they are insufficient to meet the ever-increasing demands described above. First, these profilers lack tailored support for Triton’s Just-In-Time (JIT)-compiled kernels, making it difficult to annotate kernels with their invocation contexts, specialized arguments, and tensor data type characteristics. They also fail to support customized, high-level domain-specific metrics, such as time to first token [26], bytes per second, or code region-specific latency, which are crucial for rapid, insight-driven development cycles. Lastly, they often incur significant runtime and storage overhead; for example, *nsys* produces excessively large trace databases for long-running applications, while *ncu* exhibits multi-order-of-magnitude overheads [27] when collecting comprehensive metrics, making them impractical for agile and iterative performance optimization.

Addressing these limitations requires practical yet innovative tooling solutions that balance profiling granularity with overhead while easily adapting to diverse use cases. To address this need, we present PROTON, a multi-level and adaptive profiler designed specifically for analyzing the performance of Triton kernels and end-to-end workloads built from them, with user-driven annotations and flexible configurations. PROTON addresses the aforementioned challenges through the following innovations:

- **Multi-level profiling support.** PROTON uniquely supports performance profiling at multiple programming language and compiler abstraction levels, including identifying hot regions in model inference or training, measuring execution time and hardware custom metrics of individual Triton kernels, and collecting fine-grained latencies within Triton kernels. These capabilities are designed to support the needs of all three categories of Triton users in-

troduced earlier: AI researchers, performance engineers, and compiler developers.

- **Broad usability and compatibility.** PROTON supports both command-line and Python scripting interfaces, enabling it to profile end-to-end workflows or isolate specific regions of interest. Moreover, its extensible architecture facilitates seamless integration of profiling support for both AMD and NVIDIA GPU and is designed for extension by other GPU vendors as well.
- **Low runtime and storage overhead.** PROTON integrates a lightweight runtime that supports two profiling modes: tracing and profiling. To minimize runtime overhead, it employs asynchronous communication between the PROTON’s runtime and the GPU. For storage efficiency, the profiling mode aggregates metrics by custom keys and contexts, substantially reducing data volume without sacrificing analytical utility.
- **Flexible postmortem analysis.** PROTON provides a SQL-like graph query engine that allows users to filter, merge, and compare profiling data across different contexts and runs. PROTON also supports deriving new metrics from existing ones, enabling users to conduct customized analyses. Together, these capabilities make postmortem analysis both flexible and agile.

II. BACKGROUND AND RELATED WORK

A. AI Programming Models

High-level AI programming models such as PyTorch [19], TensorFlow [28], and JAX [29] have become building blocks for AI development. These frameworks simplify programming by abstracting away low-level details, allowing users to build models without having to interact directly with the intricacies of hardware-specific implementation requirements. However, this convenience often comes at the cost of performance, particularly for non-standard models or custom operators, due to limited control over operator scheduling and hardware-specific optimizations. Other programming models, such as TVM [30], ONNX [31], and IREE [32], aim to optimize tensor expressions across the entire computation graph. While these approaches can achieve high performance, they typically involve large tuning spaces and significant engineering effort, and they may still fail to optimize certain cases. Emerging tile-based programming models [14]–[17], [33] strike a balance between high-level abstractions and low-level control. These models represent execution on GPUs and accelerators at the tile granularity, relieving users of the burden of managing individual element accesses. As a result, developers can focus primarily on algorithmic strategies for dividing tensor computation into tiles and scheduling for each tile.

B. Triton

Triton enables developers to write high-performance GPU kernels using a more accessible, Python-like syntax. An example Triton kernel is shown in Figure 1, where the kernel arguments, *x*, *y*, and *z*, are pointers to PyTorch tensors passed into the Triton kernel. The arguments *N* and *B* refer to the total

```
@triton.jit
def add_kernel(x, y, z, N, B):
    pid = tl.program_id(0)
    offsets = tl.arange(0, B) + pid * B
    x = tl.load(x + offsets, offsets < N)
    y = tl.load(y + offsets, offsets < N)
    z = x + y
    tl.store(z + offsets, z, offsets < N)
```

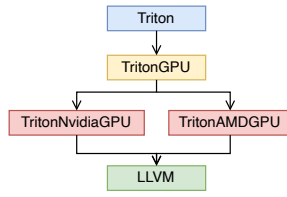


Fig. 1: Triton programming language and compiler overview

size of the tensor and the size of each tile. Users only need to specify the number of programs to execute, corresponding to parallel Cooperative Thread Arrays (CTAs) [13]; the Triton compiler handles the concurrent execution of threads and warps within each program instance. GPU micro-architectural details such as shared memory allocation and usage, as well as specialized hardware like Tensor Cores, are handled entirely by the Triton compiler. All Triton language features are exposed through the `triton.language` package (aliased as `tl`). For instance, `tl.program_id` retrieves the unique ID of a program instance, `tl.load` reads a tile from GPU global memory, and `tl.store` writes a tile back.

Each Triton kernel is JIT-compiled based on its input arguments and constants. At runtime, a hash key is computed to identify the kernel, allowing invocations with the same key to reuse the compiled binary and avoid recompilation. Triton’s compiler backend is built on MLIR [34], using a series of custom dialects to progressively lower high-level code to backend-specific assembly code. The Python frontend code is first translated into the Triton dialect and then into the TritonGPU dialect. During this process, each tile is assigned a layout that maps tensor element indices onto hardware resources such as warps, threads, and registers. Next, the TritonGPU dialect is lowered into target-specific dialects such as TritonAMDGPU or TritonNvidiaGPU, and finally converted to LLVM IR for GPU assembly code generation by their respective LLVM GPU compiler target backends: NVPTX and AMDGCN.

C. GPU Profilers

General-purpose GPU profilers [24], [35]–[39] provide tracing and profiling functionalities for GPU-accelerated applications. Tracing captures a timeline of selected events, including GPU memory transfers and kernel launches, while profiling aggregates performance metrics like instruction counts or hardware events over specific contexts. However, existing GPU profilers often lack critical DSL and framework level context information, including forward/backward associations, training iterations, and tensor metadata, which limits their effectiveness in analyzing AI workloads. To bridge this gap, AI workload-tailored profilers such as DLProf [40], the JAX profiler [41], the PyTorch profiler [42], and Amazon SageMaker Debugger [43] summarize high-level information to help users quickly identify performance-critical kernels and analyze inefficiencies. In addition, several research studies have proposed novel profiling approaches. RL-Scope [44]

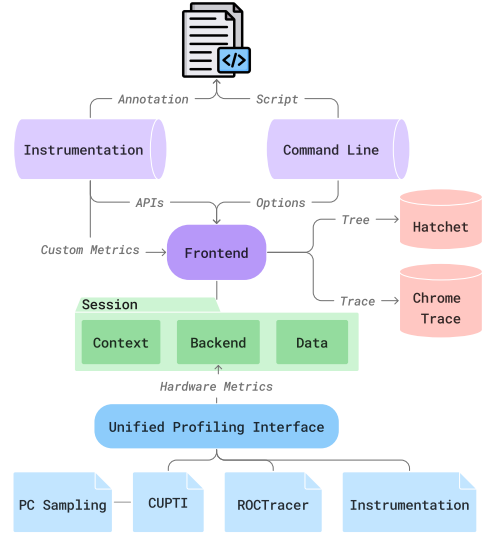


Fig. 2: Overview of PROTON’s architecture

targets reinforcement learning workloads that incur high overhead on the CPU, Hotline Profiler [45] introduces a multi-scale timeline with annotations for deep neural network training, and Skyline [46] offers an interactive profiling experience integrated into the development environment.

While existing tools can analyze workloads that include Triton kernels, none of them provide Triton-specific insights. They fail to associate necessary runtime information to identify JIT-compiled Triton kernels and lack quick, accessible diagnostics tailored to Triton’s programming and execution model.

III. OVERVIEW

A. Design

Figure 2 illustrates the core components of PROTON. Each *profiling session* encapsulates a combination of context sources, data containers, and measurement substrates. Users can initiate and terminate sessions either via command-line options or by instrumenting code with PROTON’s Python APIs. After a session is terminated, data is written to disk for postmortem analysis.

PROTON enables the collection of both user-defined and hardware-level metrics. For selected code regions or individual Triton kernels, users may define custom, domain-specific metrics, such as theoretical memory accesses, floating-point operations (FLOPs), or iteration counts. Hardware metrics are collected through a unified interface that abstracts away architectural differences across accelerator generations and vendors. Currently, PROTON integrates CUPTI [47] for NVIDIA GPUs and RocTracer [48] for AMD GPUs and is extensible to Intel GPUs via the Level Zero API [49]. PROTON also enables fine-grained instrumentation at multiple levels of IRs.

Each metric is associated with a *context* that identifies where it was recorded. These context-metric pairs are streamed into data containers that serialize the data into output formats tailored for visualization. PROTON supports two primary output formats: trace and tree. The *trace* format appends metrics to

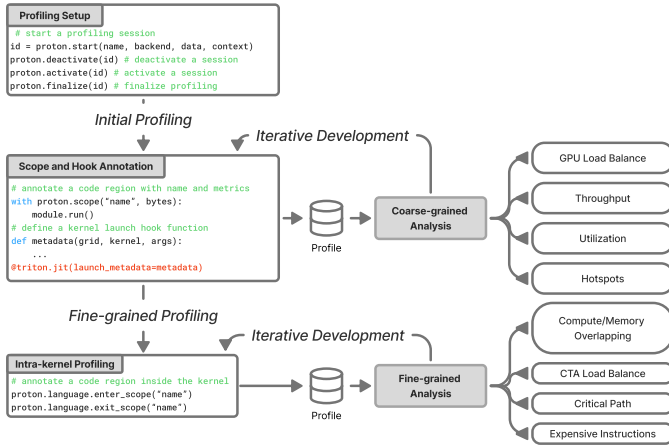


Fig. 3: PROTON's usage workflow

a timeline or stream and is compatible with Chrome trace viewers such as Perfetto [50]. The *tree* format aggregates metrics hierarchically by context and is compatible with profile visualizers like Hatchet [51] and Flamegraph [52].

B. Usage

While tools like *ncu* offer comprehensive metric coverage, they often overwhelm users seeking a quick understanding of performance bottlenecks and how to align these with expectations. Our design philosophy for APIs focuses on accelerating performance investigation tailored for Triton kernels and workloads that integrate these kernels, rather than offering an all-in-one solution. With PROTON, users are able to rapidly pinpoint a small set of multi-level correlated metrics that most directly guide optimization efforts, ensuring improvements with the greatest impact at the target level.

Figure 3 demonstrates the typical workflow of PROTON using representative APIs. The command-line options enable profiling of all GPU kernels during execution without requiring source code modifications, while the Python APIs allow users to target specific regions of interest. PROTON APIs are organized into three categories: session management, context creation, and visualization.

To set up profiling, users create profiling sessions that are identified by a unique ID returned by the `start` API and can be independently activated, deactivated, and finalized. This selective monitoring of code regions not only filters out irrelevant data from the profile but also contributes to reducing profiling overhead. For instance, for autotuned kernels generated by Triton, an initial “warmup” phase often occurs, during which a kernel is compiled using various configurations and benchmarked to select the optimal one. Tools such as *ncu* record kernels from both the warm-up phase and the main workload, imposing additional effort on users to differentiate between them. Multiple sessions can be initialized and enabled concurrently. For example, the simultaneous collection of instrumentation- and performance counter-based metrics can be conducted in a single execution with two concurrent sessions. While the command-line interface supports similar

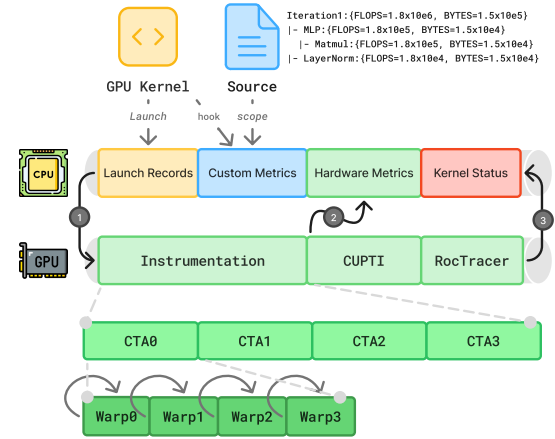


Fig. 4: PROTON's framework for metrics collection. Circled numbers indicate components of the unified profiling interface

functionality, it is limited to a single session. Hardware metrics are collected via the profiling backend and are associated with the locations where they occurred. Metrics are streamed into all data containers associated with each active profiling session.

After profiling initialization, users may annotate code regions or kernels of interest. For user-synthesized shadow context, a shadow stack is constructed to track scope entry and exit behaviors and can be associated with custom metrics, such as the number of bytes. We implement both context manager-style APIs, `scope`, and explicit function-based APIs, `enter_scope` and `exit_scope`, to accommodate different programming styles. Instead of user-based annotations, contexts can also be obtained through stack unwinding to include native and Python contexts collected using *libunwind* [53] or Python's `PyFrame_GetCode` correspondingly [54]. Different from shadow contexts, stack unwinding-based contexts use file and function names, as well as line numbers for scope names. In addition, PROTON allows users to customize kernel names and associate each kernel with custom metrics by registering the `launch_metadata` hook per kernel, without annotating every call site to the kernel.

Users can render the collected data in either a trace view or a tree view and further customize the visualization by defining text-based metric equations or filtering code regions through a SQL-like graph query language. If coarse-grained analyzes, such as throughput, utilization, or GPU load imbalance, reveal performance issues, users can iteratively update their code and observe how the metric values change. If coarse-grained metrics are insufficient, users can instrument kernels to obtain fine-grained insights that these high-level metrics cannot capture, including but not limited to, detailed instruction-level behavior (e.g., compute vs memory), parallelism, critical path, and CTA load balance.

IV. IMPLEMENTATION

A. Metrics Collection

Figure 4 illustrates the metrics collection framework. In this framework, the host is responsible for tracking the execution status of launched GPU kernels and recording user-defined custom metrics, which are stored in a host-side buffer. Hardware metrics are collected asynchronously on the GPU, independent of CPU execution. These metrics are buffered and only transferred back to the CPU when the buffer is full, signaled to the host by a GPU thread upon encountering a full device-side buffer, or when the profiling session is finalized during teardown.

1) *Custom Metrics*: PROTON supports user-defined custom metrics through two mechanisms. The first approach allows users to associate metrics with *scopes* during each invocation. The second uses *hooks* to inject metrics during kernel execution through callback functions associated with Triton’s `launch_kernel_begin` and `launch_kernel_end` handles. In these callback functions (Figure 7), users can customize kernel names, incorporating metadata such as the number of warps, threads, and shared memory usage. This metadata is stored alongside other user-defined metrics on the host. These capabilities are especially useful for analyzing autotuned kernels or kernels that are specialized at different call sites, as well as for tracking metrics that are not directly accessible via hardware counters. For instance, while the expected number of bytes moved during a matrix multiplication may correspond to the sum of the operand and result sizes from the well-known arithmetic intensity formulation, the actual data movement observed through performance counters can be higher due to effects such as L2 cache eviction. This discrepancy can lead to misleading interpretations when using hardware counters.

2) *Hardware Metrics*: Our *unified profiling interface* implements three common profiling substrates for collecting metrics using instrumentation and vendor-provided APIs, as shown Figure 4. ① represents the abstract API dispatch system, ② denotes the metric processing system, and ③ corresponds to the kernel status tracking system.

a) *API Dispatch*: Our API dispatch system simplifies development across all measurement backends, including instrumentation, RocTracer, and CUPTI. We categorize their profiling APIs into five groups: Profiling Start/End/Stop, Metric Registration/Reregistration, API Enter/Exit, Graph Capturing/Mapping/Destroy, and Buffer Allocation/Flush/Free. With such an abstraction developed, each measurement backend uses the same workflow by registering the metrics to be profiled before starting a session, allocating CPU-side buffers for storing collected data, and assigning a callback function to flush buffers and handle raw metrics. A notable divergence between the two vendors arises in instruction-level sampling. AMD supports fine-grained per-core sampling, generating a high volume of data, whereas NVIDIA aggregates samples across cores before reporting. For overhead and feasibility con-

cerns, PROTON currently only supports instruction sampling on NVIDIA GPUs.

b) *Metric Processing*: Raw metrics collected from different backends are converted into a unified set of standard metrics using shared post processing functions. This is feasible because all measurement backends expose a core set of fields relevant to our analysis. For example, kernel event records from both RocTracer and CUPTI include attributes such as the kernel name, start and end timestamps, and the corresponding graph operation when launched via a graph API. To standardize processing, we encapsulate this information in a unified `KernelMetric` structure, which abstracts away backend-specific details and simplifies downstream analysis.

c) *Kernel Status Tracking*: The kernel status tracking system is responsible for monitoring the completion of GPU kernels during profiling to trigger metrics processing and attribution. A naive approach that synchronizes the CPU and GPU after every kernel launch would introduce substantial overhead. Instead, we adopt a non-intrusive, asynchronous strategy using atomic counters to efficiently track kernel execution status. For each GPU stream, we initialize two global counters: `nSubmitted`, which tracks the number of kernels submitted for execution, and `nComplete`, which tracks the number of kernels that have completed. Upon launching a GPU kernel, `nSubmitted` is incremented. Each measurement backend spawns a background CPU thread that monitors the GPU profiling buffer usage and invokes a user-defined callback when the buffer is full. Within this callback, `nComplete` is incremented for each GPU kernel record. On the host side, metrics processing begins once `nComplete` reaches a predefined threshold—for example, after one million kernels have completed execution. When a profiling session is deactivated or finalized, PROTON will keep flushing data from the GPU to the CPU until `nComplete` matches `nSubmitted` for each stream, ensuring that all submitted kernels have been fully profiled. A similar mechanism is implemented for kernels launched by HipGraph or CudaGraph. In this case, we also maintain a mapping between individual kernels and their corresponding graph operations. `nComplete` is updated only after all constituent kernels of a graph launch have completed execution.

3) *Compiler Instrumentation*: PROTON’s instrumentation system is tailored specifically for Triton’s JIT-compilation mode and multi-level IR design.

a) *Dynamic Instrumentation*: At runtime, PROTON adopts dynamic instrumentation for JIT-compiled Triton kernels. We append the hash key associated with each Triton kernel, along with the instrumentation pattern and target architecture. This enables the runtime to distinguish between instrumented and original kernels, triggering recompilation when instrumentation is turned on. Once the user deactivates the profile session, the original kernels will be retrieved using original hashes without triggering instrumentation.

b) *Multi-level Instrumentation*: We design multi-level instrumentation dialects (i.e., `ProtonIR` and `ProtonGPUIR`) to accommodate Triton’s multi-level IR design. MLIR’s built-in

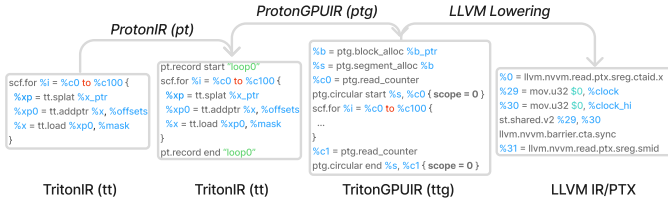


Fig. 5: PROTON's Multi-level instrumentation passes

instrumentation framework is primarily intended for compiler pass-level analysis rather than profiling, as modifying IRs using MLIR's pass instrumentation between compiler passes can lead to unintended side effects. By contrast, our dialect instrumentation is integrated into transformation passes. Additionally, we ensure that the instrumentation dialect remains transparent to the core Triton compiler transformations. In other words, the main Triton compiler passes treat our dialect as an unknown dialect, without needing to import or depend on it explicitly.

Figure 5 illustrates how Triton IRs can be instrumented to measure the time spent in a loop. In Proton (pt), we define a single operation, `pt.record`, which corresponds directly to the frontend scope API. In ProtonGPU (ptg), `ptg.record` is replaced with operations that interact with hardware counters and annotate warp segments. Static scope name resolution is also performed to map scope strings to unique integer identifiers, minimizing runtime overhead. Additionally, we use IRs, such as `block_alloc` and `segment_alloc`, to manage profiling buffers. Finally, the dialect is lowered into LLVM IR and backend-specific assembly code.

c) Buffer Management: To collect profiling data generated by instrumented operations, PROTON manages its own profiling buffer on the GPU. In the default mode, the size of the profiling buffer equals the total number of CTAs to be profiled. PROTON also incorporates sampling modes that allow users to monitor a single warp, a subset of warps, or a selected set of CTAs for reduced overhead and data size. As shown in Figure 4, each CTA has its own “block” of dedicated buffer memory to store profiling data. Each block is then subdivided into “segments” accessed by individual warps. Among the various buffer types we support, the default is a “circular” buffer per warp: when a warp reaches the end of its segment, it wraps around to the beginning. This enables continuous monitoring over a window of execution. While there is some risk of overwriting data if the buffer size is set too small, this approach in practice at least provide a high fidelity trace of certain time interval. In addition to a circular buffer, PROTON also integrates “synchronized” buffers to periodically flush data to the CPU, preventing the loss of data but may introduce slightly higher overhead.

B. Metrics Attribution

In this subsection, we describe how the collected metrics are associated with specific contexts spanning CPUs and GPUs and are attributed to different data containers.

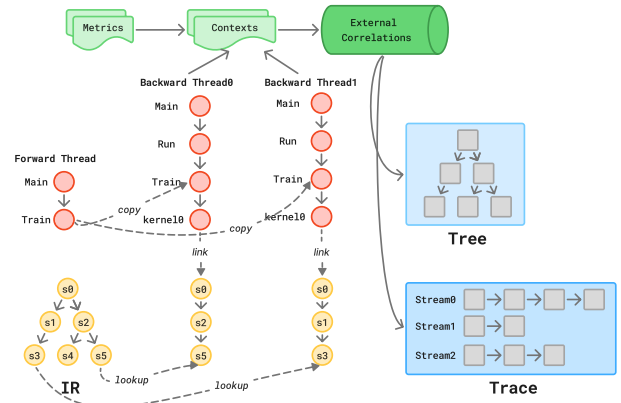


Fig. 6: Metric attribution on a system equipped with a CPU and two GPUs using shadow context

a) Host Context: We define a context as a path consisting of nested scopes that identify where a metric originates. This path can be a call stack obtained using unwinding APIs, but it often tends to be long, especially in deep learning applications, where GPU kernels are invoked deep within stacks of Python and C++ function calls.

To address this, we introduce a “shadow” context mechanism. As shown in Figure 6, a shadow stack is initialized on the CPU thread (Forward Thread) that initiates profiling. At each scope entry (`enter_scope`), we push a scope onto a thread-private stack; we pop it on scope exit (`exit_scope`). Each GPU operation is associated with scopes of the stack when it is invoked from the CPU as its context. When a new thread is created, we copy the current context from the main thread and continue pushing new scopes independently. In PyTorch, the autograd module creates a number of threads equal to the number of GPU devices involved in backward propagation to handle backward computation in parallel. In this case, the shadow contexts of backward threads represent the logical view of execution from the programmer's perspective.

b) Device Context: Obtaining device-side contexts differs significantly because no unwinding APIs or data structures are available on the GPU. To recover the context on the GPU device, we devise static analysis using user-provided scope annotations. We traverse all scopes within a function in breadth-first order and assign each context (e.g., `s0`→`s2`) a unique identifier. Postmortem, we recover the scope names belonging to each context from the previous analysis results. This approach supports device-side function calls, including inlined and nested calls, as long as there are no recursive or mutually recursive functions.

c) Data Container: We associate a metric with its corresponding context before attributing it to a data container. At the point where a GPU kernel is launched, an external correlation ID (i.e., `nSubmitted` in subsection IV-A) is recorded and associated with the host-side context of the kernel being launched. All metrics triggered with this specific launch event are associated with this correlation ID. When these metrics

are returned from the GPU buffer, a lookup is done through the correlation/context map to find the host context and link it with each metric’s device context.

PROTON supports two data container types: traces and trees. The *trace* container maintains a timeline view organized by GPU streams. Metrics triggered within the same stream are appended sequentially. The *tree* container represents a hierarchical execution structure. Nodes may correspond to source lines, compiler-generated scopes, or user-defined scopes. Furthermore, metrics in the tree container are aggregated according to their type. Each metric is first assigned to the corresponding tree node using its associated context. Then, metrics are aggregated based on their types defined as follows:

- *Inclusive*: Aggregated at a node and propagated upward to parent nodes. Example: elapsed time.
- *Exclusive*: Aggregated only at the current node and not propagated. Example: amount of shared memory used by a kernel launch.
- *Property*: Metadata attached to a node but not aggregated. Example: kernel name.

C. Visualization & Analytics

We describe the visualization and analytic functionalities in this section and present concrete views in section V.

PROTON integrates both Hatchet [51] and Chrome Trace format, both of which are lightweight and efficient. The hatchet-viewer outputs a top-down tree structure, color-coded to highlight performance hotspots, allowing users to drill down and explore nested contexts. Users can customize the tree using command-line queries for metric derivation, differential analysis, and tree structure filtering. Users can also define derived metrics by supplying simple equations (e.g., $a/\min(b, c)$); the viewer then constructs an abstract syntax tree (AST) to parse and evaluate these expressions at each node, updating the corresponding metrics as necessary. This capability is especially useful for postmortem computation of derived metrics such as roofline efficiency and bandwidth utilization. Differential analysis is implemented by subtracting the same metrics from matching nodes between two trees after alignment following the contexts, which enables rapid comparison and exploration of impacts due to kernel code updates. For large profiles, PROTON integrates a graph query engine that enables users to filter or focus on specific subtrees based on query constraints.

In the trace view, users can inspect either coarse-grained kernel-level metrics or fine-grained events within individual kernels. In the former case, each line represents a GPU stream consisting of sequential kernel events. In the latter, each line corresponds to a CTA within a kernel, showing metrics captured on the device side. These metrics are annotated with their associated scope names and are linked to the host-side context if available.

V. USE CASES

```
1 def _matmul_launch_metadata(grid, kernel, args):
2     ret: Dict[str, Union[float, int]] = {}
3     M, N, K = args["M"], args["N"], args["K"]
4     var = args["VARIANT"]
5     elem_bytes = args["c_ptr"].element_size()
6     ret["name"] = f"{kernel.name}{var}"
7     ret[f"flops{elem_bytes * 8}"] = 2.0 * M * N * K
8     ret["bytes"] = elem_bytes * (M * K + N * K + M * N)
9     return ret
10
11 with triton_prof.scope(f"[M={M}, N={N}, K={K}]"):
12     ...
```

Fig. 7: Using hook and scope to annotate metadata

```
1 data = graph.filter(f"MATCH ('*', c) WHERE c.'name' =~
2     ↳ '{op_regex}' AND c IS LEAF").dataframe
3 bytes = int(data["bytes"].sum())
4 flops = int(sum(data[[c for c in ["flops8", "flops16"] if
5     ↳ c in data.columns]].sum()))
```

Fig. 8: Postmortem extraction of flops and bytes

A. Iterative Kernel Development and Optimization

A deep learning operator, such as matrix multiplication, can be implemented with different algorithm variants depending on hardware features and input characteristics. On NVIDIA GPUs, for example, these variants include Tensor Memory Access [55] (TMA), persistent processing [56], and warp specialization [57]. Each variant can further be autotuned with configurations such as block sizes and warp counts. In practice, this leads to a workflow of exploring these configurations and algorithm variants under different precisions and input shapes, benchmarking them against the best-known implementations (e.g., cuBLAS [58]), and iteratively refining custom kernels by improving the code and the compiler. **However, general profilers like *ncu* and *nsys* fall short in this setting as they lack the dedicated infrastructure to differentiate between kernel autotuning and execution phases, cannot annotate kernels with different input configurations, do not capture the input shapes associated with each kernel launch, and may introduce prohibitive overhead.**

To address these limitations, we leverage PROTON’s *scope* and *hook* utilities to annotate kernels and aggregate metrics. Figure 7 illustrates our implementation. We rename each kernel by updating its original name using the algorithm variant and associate it with both the FLOPs and bytes metrics. Kernels launched with the same input shapes are grouped together using *scope*. Postmortem, we filter irrelevant kernels and extract each relevant kernel’s FLOPs and bytes using PROTON’s graph query utility, as shown in Figure 8. These capabilities allow us to compare custom Triton kernels against vendor library baselines and track performance gains following compiler and algorithmic improvements. Figure 9 shows a representative terminal output. Without loop flattening, the best-performing kernel achieves 1072.4 TFLOPS. After applying loop flattening to merge nested loops, the *tma_persistent*

	diff	origin	+flatten	kernel
[M=8192, N=8192, K=512, Prec=fp8]				
-	-2.2	1167.8	1165.6	cublas
-	-0.2	910.6	910.3	matmul_kernel
-	+96.3	941.2	1037.6	matmul_kernel_persistent
-	+2.5	1072.4	1075.0	matmul_kernel_tma
-	+231.6	938.9	1170.6	matmul_kernel_tma_persistent

Fig. 9: Performance comparison in TFLOPS showing timing differences across matrix multiplication kernels on NVIDIA GH200

variant improves by 231 TFLOPS, reaching 1170.6 TFLOPS, matching or exceeding the cuBLAS performance of 1165.6 TFLOPS.

B. Continuous Integration

PROTON is integrated into the regression testing of our Triton benchmark suite that consists of a range of widely used operators, such as TopK, Sort, MoE, Attention, Swiglu, etc. The benchmarks are designed to cover a broad set of high impact kernels that address both bandwidth and compute bound scenarios. The integration of PROTON serves two purposes. First, to evaluate the level of hardware utilization and track performance regression; and second, to compare performance across GPU vendors, specifically AMD and NVIDIA.

a) Single GPU: We use PROTON hooks to calculate the theoretical number of bytes and FLOPs incurred by each operator. Next, we derive the arithmetic intensity of each operator to determine whether the operator is compute- or bandwidth-bound and compute its utilization metrics relative to the theoretical peaks. We consider two types of peaks: (1) the hardware-specified peak and (2) the practical peak measured by the sustained FLOPs/bandwidth achieved by a well-optimized vendor-provided library. The second is crucial because modern GPUs are power-constrained, and sustaining a fixed maximum frequency is not feasible. Thus, comparing custom operators (e.g., MoE) against a related implementation in vendor libraries such as `matmul` provides a more realistic sense of the achievable peak. On AMD and NVIDIA GPUs, we measure practical peaks using hipBLAS and cuBLAS, respectively, using the `scope` utility. **In contrast, existing tools such as `ncu` and `rocprow` can only measure architecture-specific peaks using metrics available through exposed hardware counters.**

We also use different `scopes` to record arithmetic intensity under varying workloads (e.g., increasing the batch size) and plot the results in Figure 10a and Figure 10b, which show roofline models [59] on AMD and NVIDIA GPUs, respectively. Both figures are measured using the same MoE implementation, sharing the same PROTON hook.

From the figures, we observe that the MI300X’s realistic FP8 performance on compute-intensive operators falls far below its claimed 2.6 PFLOPs peak. Even well-optimized hipBLAS kernels achieve less than 50% of the hardware peak. By contrast, cuBLAS on GH200 sustains up to 64% of peak. Further, our MoE operator reaches 84% of the cuBLAS

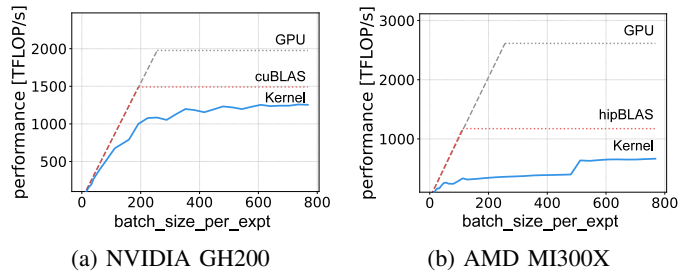


Fig. 10: Comparison of roofline with varying batch size per expert. The number of experts is fixed as four. The precision we use is FP8

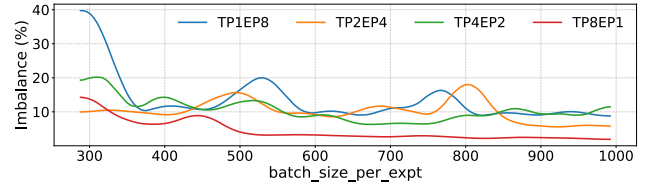


Fig. 11: Compute imbalance under different parallelism strategies on a single node with eight AMD MI300X GPUs.

peak on GH200, but only 56% of the hipBLAS peak on MI300X. This gap is potentially caused by missing compiler optimizations in Triton’s AMD backend.

b) Multi-GPU: PROTON’s compact profile makes it feasible to analyze multi-GPU scenarios, where the profile size scales with the number of processes/GPUs. Each PROTON profile is only up to hundreds of kilobytes, as we will show in the Evaluation section, while other tools can yield significantly larger profiles. In our multi-GPU MoE test, we employ tensor parallelism (TP) and expert parallelism (EP) to distribute work across devices and dump a profile per GPU. Figure 11 presents an imbalance analysis across multiple GPUs. We define the *compute imbalance factor* as the relative gap between the least and most utilized GPUs:

$$\text{Compute Imbalance} = 1 - \frac{\min_i \text{FLOPS}_i}{\max_i \text{FLOPS}_i}.$$

A higher value indicates a more severe token distribution imbalance across experts. We observe that TP=8 consistently yields lower imbalance than other policies. Moreover, as long as $\text{EP} \neq 1$, different policies show comparable compute imbalance when the batch size is small.

C. Performance Characterization of LLM Inference Engines

We demonstrate how PROTON aids not only in analyzing individual Triton kernels but also in diagnosing performance issues in end-to-end deep learning applications that incorporate both Triton and CUDA/HIP kernels, such as large language model (LLM) inference engines [60], [61]. LLM inference typically consists of *prefill* and *decoding* stages; understanding various latency and throughput metrics associated with each stage and decomposing the runtime into specific kernels is

[Model=Llama-3.1-8B-Instruct QPS=500 Tokens/s: 9281]			
phase/node	TBPS (tbyte/s)	GPU (ms)	CPU (ms)
+ prefill	0.35	28653	29300
+- self-attn	0.59	8444	27572
- attn	1.26	2946	1803
- qkv_proj	0.21	2926	286
- o_proj	0.21	2199	376
- rotary_emb	0.89	319	227
+- mlp	0.25	19782	1091
- act_fn	1.08	1087	254
- down_proj	0.21	6331	312
- gate_up_proj	0.20	12302	289
+ decoding	1.19	6489	9418
+- self-attn	1.38	2914	7030
- attn	1.62	1938	2079
- qkv_proj	1.17	445	404
- o_proj	0.88	399	515
- rotary_emb	0.42	65	318
+- mlp	1.07	3458	1550
- act_fn	0.58	192	380
- down_proj	1.15	1043	425
- gate_up_proj	1.08	2226	417

Fig. 12: Performance characterization of Llama-3.1-8B-Instruct on NVIDIA A100

essential. PROTON is well-suited for this task due to its fine-grained control of profiling and scope-based metric collection. activate and deactivate APIs can be used to skip lower interest regions, such as initial data processing and kernel warmup, without introducing storage and runtime overhead. `scope(..., metrics)` can be easily extended to collect CPU time and derived metrics, including, but not limited to, tokens per second (TPS), time to first token (TTFT), and time per output token (TPOT) for performance characterization of different models and patterns (e.g., thinking/reasoning or agent tool use). **In comparison, end-to-end tools such as *nsys* with NVTX-based annotations [62] cannot skip uninteresting regions, are limited to NVIDIA platforms, and cannot be used for custom metric aggregation**, making them less suited for quick performance characterization.

Figure 12 illustrates GPU time, CPU time, and memory throughput across different inference phases, profiled on an NVIDIA A100 GPU using vLLM [61] and the Sonnet dataset [63]. We observe high CPU overheads in the `self-attn` module. In particular, only 8444 ms is spent on the GPU, while 27572 ms is spent on the CPU in the prefill phase. This discrepancy indicates that applying CUDAGraph for this module to merge kernel launches is beneficial. We also note that the output projection (`o_proj` and `qkv_proj`) in the `self-attn` module is suffering from low memory throughput, indicating potential memory access optimization opportunities.

D. Intra-kernel Tracing and Profiling

To demonstrate PROTON’s capability for fine-grained profiling, we instrumented semantic scopes in a Triton-based FlashAttention [20] kernel with warp specialization enabled. Each CTA in this kernel employs five warps: one dedicated to GEMM, two for softmax computation, and two for data movement.

TABLE II: Experiment platforms

Accelerator	GPU Memory	Compute Units	Software Stack
GH200	96 GB (4 TB/s)	132 SMs	CUDA 12.6
MI300X	192 GB (5.3 TB/s)	304 CUs	ROCm 6.4

Figure 13b shows the breakdown of normalized cycles across CTAs. The profile view shows the calling contexts to warp and computation stages with CPU and GPU scopes. Figure 13a further exposes concurrent execution of CTAs and warps. These two views **highlight the full calling contexts and asynchronous behavior that existing tools cannot capture**. In the trace view, the upper pane shows an uneven computational workload across CTAs. This imbalance arises because the workload is parallelized by tokens, while each token may use different causal masks. Persistent attention, which enables idle CTAs to take over remaining work, could provide better resource utilization and improve overall performance. Expanding a CTA tab reveals individual warp timelines, where each line corresponds to a warp and is composed of stacked colored regions denoting semantic activities.

The softmax warp is decomposed into two computation stages: *off-band* computation (i.e., `stage1`) followed by *on-band* computation (i.e., `stage2`) induced by causal masking. Each stage consists of four semantic regions: `qk_load`, `row_max`, `exp2`, and `reduce`. Among these, the `exp2` stage is particularly costly. When multiple warps invoke `exp2` simultaneously, contention for the GPU’s Special Function Units (SFUs) causes high stall rates. To address this, we restructure the `exp2` computation into finer-grained sub-stages that interleave `exp2` with shared memory loads. This optimization alleviates SFU contention and improves FlashAttention performance by up to 20% on NVIDIA GB200 across various context lengths and hidden dimensions.

VI. EVALUATION

We measure the overhead of PROTON with the following experiments:

- End-to-end model profiling: Measures the runtime and storage overheads of PROTON for profiling end-to-end models consisting of multiple Triton kernels on AMD and NVIDIA GPUs and compares them with vendor profilers and the PyTorch profiler (TorchProf).
- Intra-kernel profiling: Measures the runtime overhead incurred by intra-kernel instrumentation within individual Triton kernels on AMD and NVIDIA GPUs.

The hardware and software configurations used in experiments are listed in Table II.

A. End-to-End Profiling

We evaluate a diverse set of open-weight language models spanning different scales, precisions, and modalities. All models come from Unsloth [64], which provides high-performance Triton implementations. The collection includes dense models

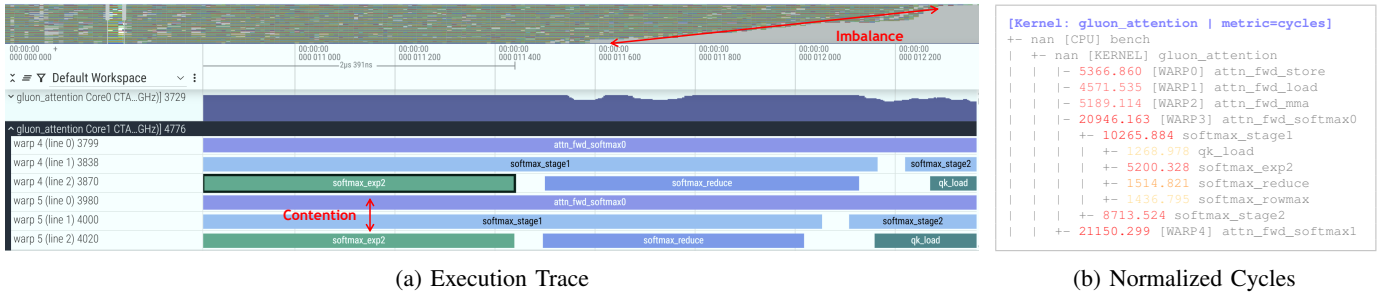


Fig. 13: Intra-kernel tracing and profiling views of an attention kernel on an NVIDIA GB200 GPU

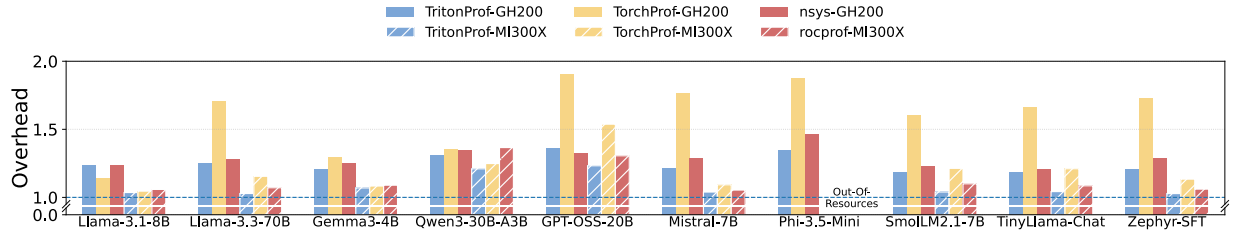


Fig. 14: End-to-end profiling overhead on open-weight LLMs

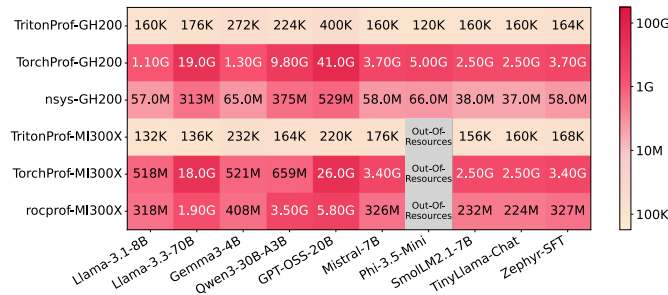


Fig. 15: Size of profiles collected by each tool

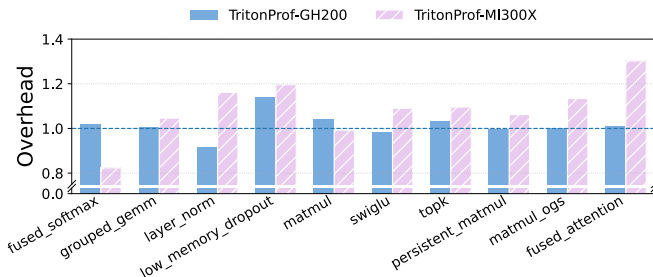


Fig. 16: Intra-kernel profiling overhead on various kernels

such as Llama-3.1-8B [65], [66], Llama-3.3-70B (with 4-bit QLoRA) [67], Gemma3-4B (text+vision) [68], Mistral-7B-Instruct [69], Phi-3.5-Mini-Instruct [70], SmolLM2.1-7B-Instruct [71], [72], TinyLlama-Chat [73], and Zephyr-SFT [74], [75], as well as mixture-of-experts (MoE) models including Qwen3-30B-A3B [76] and GPT-OSS-20B [77]. Most models run in BF16 precision, while several employ quantization (e.g., 4-bit QLoRA for Llama-3.3-70B and Qwen3-

30B-A3B). All models are runnable on NVIDIA GH200 and AMD MI300X GPUs, except for Phi-3.5-Mini-Instruct, which encounters a Triton out-of-resources error.

We compare PROTON against *nsys* on GH200, *rocprof* on MI300X, and the PyTorch Profiler on both platforms. For a fair comparison, all profilers are restricted to collecting GPU events only. Each workload is profiled with 30 training steps using standard fine-tuning configurations and inputs. Runtime overhead is reported as the average step time relative to execution without profiling, and storage overhead is measured as the size of the profile file produced by each tool.

Figure 14 reports the runtime overheads incurred by each tool. PROTON introduces the lowest average overheads among all profilers: $1.25\times$ on GH200 and $1.08\times$ on MI300X. In contrast, on GH200, *nsys* incurs an average overhead of $1.29\times$, and TorchProf triggers $1.60\times$. On MI300X, *rocprof* introduces a $1.13\times$ average overhead, and TorchProf imposes $1.19\times$. Overall, TorchProf yields the largest overhead across workloads. An exception occurs with Llama-3.1 on GH200, where the workload is susceptible to CPU-side performance issues. Even enabling the profiler without data collection leads to high overhead.

Figure 15 compares profile sizes across tools. While other profilers must record every event in the trace, PROTON’s profile mode stores only aggregated data. As a result, PROTON produces compact profiles of only a few hundred bytes, yet still delivers sufficient insights for all use cases. By contrast, other tools produce profiles $231\times$ to $132,352\times$ larger than PROTON. TorchProf generates the largest profiles, reaching up to 41 GB. Such sizes make profiling long-running applications impractical and add overhead to rapid kernel development or continuous integration workflows.

B. Intra-kernel Profiling

To measure the runtime overhead of PROTON with intra-kernel instrumentation, we study ten widely used Triton kernels from its tutorials and official benchmarks. Each kernel is instrumented with scopes at function entry/exit, loop boundaries, setup, and finalization to capture the time spent in these stages. We use the default “circular” buffer type without sampling. Kernel runtimes on both MI300X and GH200 are measured by recording GPU events before and after kernel launches, with timestamp differences computed using the sleep trick [78] for high fidelity.

Figure 16 presents the intra-kernel profiling overheads. PROTON imposes average overheads of $1.01\times$ on GH200 and $1.09\times$ on MI300X. Although the average overhead is low and demonstrates the high fidelity of PROTON, we observe two noteworthy phenomena. First, the `fused_attention` kernel exhibits significantly higher overhead on MI300X than on GH200 under the same configuration. This is due to differences in L1 cache size between the two GPUs: GH200 provides a 256 KB L1 cache per SM, whereas MI300X has only 32 KB per CU. PROTON reserves a portion of the L1 cache (shared memory) for storing profiling data; hence, kernels running on GPUs with smaller physical L1 sizes are more likely to experience performance degradation. Second, for kernels such as `low_memory_dropout`, the overheads are above $1.10\times$. PROTON applies heuristics to select the largest feasible buffer size that avoids data loss while minimizing L1 cache usage. Currently, buffer sizes are limited to powers of two, which may cause the reservation of unnecessary profiling buffers and reduce the available L1 cache for the GPU kernel; relaxing this constraint could further reduce overhead. Noting that we observe overheads in some cases that are less than 1.0, such as layer normalization and swiglu kernels on GH200, this can be caused by instruction scheduling, especially for those small kernels.

VII. DISCUSSION

A. Profiling Overhead

For end-to-end profiling, the overhead of PROTON primarily depends on the cost introduced by the measurement substrates, such as the RocTracer API and the CUPTI API. PROTON’s runtime itself introduces minimal overhead and only correlates GPU activities with their corresponding host launch APIs.

For intra-kernel profiling, the instrumentation mode depends on the number of instrumented scopes, which increases overhead due to additional instructions, shared-memory accesses, and reduced available L1 cache. For the instruction sampling mode, we compared the overhead of `ncu` in PROTON in the final version and found that NCU’s overhead can be up to $20\times$ higher than PROTON’s, even when collecting the same set of metrics and without kernel replay. We are unable to identify the root cause because `ncu` is closed-source, but we have reported the issue to NVIDIA.

B. Profiling Accuracy

For end-to-end profiling, the accuracy of PROTON depends on the vendor-provided APIs used to collect hardware metrics. For intra-kernel profiling with instruction sampling, the accuracy depends on the sampling frequency, which can be configured by the user. For intra-kernel profiling with instrumentation, the average latency of hardware timing instructions on NVIDIA GPUs is only a few cycles, while on AMD GPUs, we observe that these timing instructions can take tens of cycles.

C. Scope Annotations

PROTON provides both the `scope` and `hook` utilities to associate code regions with user-defined names. The `scope` utility does not rely on any specific integration and can work with any framework. The `hook` utility works as long as the framework uses Triton’s default kernel-launch driver, which is the case for TorchInductor [19] and JAX [18].

We recognize this limitation and plan to explore the potential of using AI to automatically generate scopes and metrics. For example, the system prompt provided to LLMs could include several examples of using PROTON for annotations; based on the capabilities of modern AI agents, we believe they can understand code and generate annotations automatically. However, we also recognize that it is ultimately the user’s responsibility to verify that the computation of user-defined custom metrics is correct.

VIII. CONCLUSIONS

In this paper, we present PROTON, a multi-level, adaptive, and low-overhead profiler for Triton that brings unprecedented usability for practical workloads and visibility into GPU kernel execution. Through comprehensive case studies and benchmarks, we demonstrated that PROTON achieves low runtime and storage overhead while providing easy-to-use interfaces to gain performance analysis and optimization insights on production-grade AI kernels. Looking forward, we plan to enhance PROTON’s profile view with an interactive GUI using frameworks such as Textual [79]. We also plan to integrate profiling feedback directly into Triton’s compiler’s optimization passes.

DATA-AVAILABILITY STATEMENT

The artifact associated with this paper is publicly available on Zenodo [80].

ACKNOWLEDGMENT

We thank the reviewers for their valuable feedback on improving the paper. We also thank AMD for fixing bugs in the AMD backend. This work was supported by NSF grant NSF-2411134 and a donation from AIGCSEMI LLC. The computing resources are provided by AMD Developer Cloud and lambda.ai.

A. Abstract

We present PROTON, a multi-level, adaptive profiler tailored for the Triton programming language and compiler. The submitted artifact consists of 2 parts:

- The source code of PROTON implementation.
- The scripts demonstrate the usage of PROTON and reproduce the key results presented in the paper.

The artifact aims to reproduce the results in sections 5 and 6.

B. Artifact check-list (meta-information)

- **Run-time environment:** CUDA 13.0 for the NVIDIA platform; ROCm 6.4 for the AMD platform.
- **Hardware:** GH200 for the NVIDIA platform; MI300X for the AMD platform. We will provide access to them for the reviewers.
- **Execution:** Provided shell and python scripts.
- **Metrics:** Measured profiling overheads.
- **Output:** JSON data, CSV table, and visualized figures.
- **Experiments:** Reproduction of Figures 9, 10, 11, 13, 14, 15, and 16.
- **Disk space required:** at least 200 GB.
- **Time needed to prepare the workflow:** 30 minutes
- **Time needed to complete experiments:** 9 hours
- **Publicly available?:** Yes
- **Code licenses (if publicly available?):** MIT
- **Data licenses (if publicly available?):** MIT
- **Workflow framework used?:** PyTorch
- **Archived (provide DOI?):** <https://doi.org/10.5281/zenodo.17637318>

C. Description

We briefly provide the instructions in this artifact appendix. For detailed step-by-step instructions, please follow the README.md in the given GitHub repository.

1) *How delivered:* The artifact is publicly available at GitHub: <https://github.com/Deep-Learning-Profiles-Tools/CGO-26-AE>.

All the required code can be obtained from the Git repository.

2) *Hardware dependencies:*

- 1× GH200 GPU for the NVIDIA platform.
- 1× MI300X GPU for the AMD platform.

3) *Software dependencies:* Basic software dependencies include:

- CUDA 12.8 for NVIDIA / ROCm 6.4 for AMD.
- GCC 12 or newer.
- uv python package manager.

We manage the Python environment through uv. You can look up the full environment dependencies in `pyproject.toml`.

D. Installation

First, clone the Git repository and initialize the Git submodules.

- `git submodule update -init -recursive`

Then, we set up the python virtual environment with uv. First, install uv by

- `curl -LsSf https://astral.sh/uv/install.sh | sh`
- `source $HOME/.bashrc` to refresh your bash environment or open a new terminal

Then we can set up the python environment with uv.

- `export UV_SKIP_WHEEL_FILENAME_CHECK=1`
- `uv sync -extra cuda` for the NVIDIA platform.
- `uv sync -extra rocm` for the AMD platform.

E. Experiment workflow

We provide a one-click shell script for each experiment to reproduce it easily. The experiments are as follows:

1) Major Results:

a) *Figure 14 and Figure 15:* Figure 14 shows the end-to-end profiling overhead during the training of open-weight LLMs with unsloth. Figure 15 shows the comparison of profile sizes collected by Proton and vendor-provided profiling tools. You can reproduce these two figures using the following workflow:

- 1) `cd fig14_15`
- 2) Run experiments:
 - NVIDIA: `python run_experiment_nv.py`
 - AMD: Due to rocprof incompatibility with ROCm 7+, first downgrade to ROCm 6.4:
 - `uv pip uninstall torch torchvision triton`
 - `uv pip install torch torchvision --index-url https://download.pytorch.org/whl/rocm6.4`
 - `uv pip install triton==3.4.0`

Then run: `python run_experiment_amd.py`
After completion, restore the environment:

- `cd ..`
- `uv sync -extra rocm`

The results will be displayed in the terminal and stored in a JSON file.

3) `python fig14.py` to generate Figure 14. The script will automatically read the JSON file and plot the corresponding portions of the figure (i.e., the NVIDIA results on NVIDIA GPUs and the AMD results on AMD GPUs).

4) `python fig15.py` to generate Figure 15. The procedure is similar to that for Figure 14.

5) Check `fig14.pdf` and `fig15.pdf`.

b) *Figure 16:* Measurement of intra-kernel profiling overheads by comparing timings profiled with the CUPTI backend between scenarios with and without intra-kernel instrumentation.

- 1) `cd fig16`
- 2) `bash run_exp_and_visualize.sh`
- 3) Check the results in the CSV table under the folder `fig15/kernels`
 - NVIDIA: `cupti_profile_timings_nv.csv`
 - AMD: `cupti_profile_timings_amd.csv`
- 4) Visualize the results by `python fig16.py` and check the generated figure named `fig16.pdf`.

2) Optional Results:

a) *Figure 9:* Performance comparison in TFLOPS shows timing differences across matrix multiplication kernels.

- 1) `cd fig9`
- 2) `bash run.sh`
- 3) The proton profile will be visualized and printed in the terminal output.

b) *Figure 10:* Comparison of the roofline with varying batch sizes per expert in the MoE kernel.

- 1) `cd fig10`
- 2) `bash run.sh`
- 3) Check results under `fig9/triton_kernels/bench/logs/gpt-oss-x2`

c) *Figure 11:* Compute the imbalance under different parallelism strategies on a single node with eight MI300X GPUs. NOTE: this experiment does not work with ROCm 7.0 due to ROCm bugs, and it requires the user to install ROCm 6.4.

- 1) `cd fig11`
- 2) `bash run.sh`
- 3) Check the results under `fig10/triton_kernels/bench/logs/gpt-oss-x2`

d) *Figure 13*: Intra-kernel tracing and profiling views of an attention kernel.

- 1) `cd fig13`
- 2) `bash run.sh`
- 3) Similar to figure 9, the proton profile will be visualized and printed in the terminal.
- 4) Drag the generated `gluon_attention_trace.chrome_trace` to the Peretto UI to view the visualized trace.

F. Evaluation and expected result

Reproduce the major results, including Figures 14, 15, and 16. They should be close to the figures in the manuscript. Optionally, also reproduce Figures 9, 10, 11, and 13.

G. Experiment customization

In each experiment sub-folder, we provide a detailed explanation of how the experiment is organized and performed so that it can be further customized.

H. Methodology

Submission, reviewing, and badging methodology:

- <http://cTuning.org/ae/submission-20190109.html>
- <http://cTuning.org/ae/reviewing-20190109.html>
- <https://www.acm.org/publications/policies/artifact-review-badging>

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, vol. 30, 2017.
- [2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [3] J. Karra, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4401–4410.
- [4] A. Kirillov, E. Mintun, N. Ravi, H. Mao, L. Rolland, L. Gustafson, Y. Xiao, S. Whitehead, A. Mirhoseini, R. Mottaghi *et al.*, "Segment anything," *arXiv preprint arXiv:2304.02643*, 2023.
- [5] A. Radford, J. W. Gao *et al.*, "Robust speech recognition via large-scale weak supervision," *OpenAI technical report*, 2022, <https://cdn.openai.com/papers/whisper.pdf>.
- [6] E. Hernandez *et al.*, "Current designs of robotic arm grippers: A comprehensive systematic review," *Sensors*, vol. 23, no. 3, p. 1136, 2023.
- [7] Y. Huang, J. Zhang *et al.*, "Indoor positioning systems of mobile robots: A review," *Sensors*, vol. 23, no. 4, p. 1821, 2023.
- [8] P. Eksombatchai, P. Jindal, J. Liu, J. Sharma, R. Mehrotra, and C. Tan, "Pixie: A system for recommending 3+ billion items to 200+ million users in real-time," in *Proceedings of the 2018 World Wide Web Conference*, 2018, pp. 1775–1784.
- [9] S. Markidis, S. W. D. Chien, E. Laure, I. B. Peng, and J. S. Vetter, "Nvidia tensor core programmability, performance & precision," *arXiv preprint arXiv:1803.04014*, 2018.
- [10] AMD Corporation, *AMD CDNA™ 3 Architecture White Paper*, 2024, <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf>.
- [11] N. Corporation, "Nvidia h100 tensor core gpu architecture," <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>, 2022.
- [12] NVIDIA Corporation, *NVIDIA Blackwell Architecture Technical Brief*, 2024, <https://resources.nvidia.com/en-us-blackwell-architecture>.
- [13] —, *CUDA C++ Programming Guide*, NVIDIA, 2023, <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>.
- [14] P. Tillett *et al.*, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019.
- [15] Z. Jiang *et al.*, "Cutile: A compiler-assisted tiled programming abstraction for gpus," *arXiv preprint arXiv:2302.01234*, 2023.
- [16] Anonymous, "Tilelang: Language-level support for optimizing tiled computation," *arXiv preprint arXiv:2305.01234*, 2023.
- [17] L. Chen *et al.*, "Thunderkittens: A domain-specific language for large-scale ai on gpus," 2024, to appear.
- [18] JAX Team, *JAX Pallas GPU Programming Interface*, 2024, <https://docs.jax.dev/en/latest/pallas/gpu/index.html>.
- [19] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski *et al.*, "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 929–947.
- [20] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in neural information processing systems*, vol. 35, pp. 16 344–16 359, 2022.
- [21] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," *arXiv preprint arXiv:1701.06538*, 2017.
- [22] H. Lee, G. Park, J. Suh, and S. J. Hwang, "Infinitehip: Extending language model context up to 3 million tokens on a single gpu," *arXiv preprint arXiv:2502.08910*, 2025.
- [23] P.-L. Hsu, Y. Dai, V. Kothapalli, Q. Song, S. Tang, S. Zhu, S. Shimizu, S. Sahni, H. Ning, and Y. Chen, "Liger kernel: Efficient triton kernels for llm training," *arXiv preprint arXiv:2410.10989*, 2024.
- [24] NVIDIA Corporation, *NVIDIA Nsight Systems*, 2025, version 2025.3.1. [Online]. Available: <https://developer.nvidia.com/nsight-systems>
- [25] —, *NVIDIA Nsight Compute*, 2023, version 2023.2, <https://developer.nvidia.com/nsight-compute>.
- [26] —, *Metrics — NVIDIA NIM LLMs Benchmarking*, 2024, accessed: 2025-05-20. [Online]. Available: <https://docs.nvidia.com/nim/benchmarking/llm/latest/metrics.html>
- [27] Y. Hao, N. Jain, R. Van der Wijngaart, N. Saxena, Y. Fan, and X. Liu, "Drpup: A top-down profiler for gpu applications," in *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*, 2023, pp. 43–53.
- [28] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "{TensorFlow}: a system for {Large-Scale} machine learning," in *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, 2016, pp. 265–283.
- [29] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: Composable transformations of python+numpy programs," <https://github.com/google/jax>, 2018, accessed: 2025-05-20.
- [30] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 578–594. [Online]. Available: <https://arxiv.org/abs/1802.04799>
- [31] ONNX Community, "Open neural network exchange (onnx)," <https://github.com/onnx/onnx>, 2019, accessed: 2025-05-20.
- [32] IREE Team, "Iree: Intermediate representation execution environment," <https://github.com/openxla/iree>, 2023, accessed: 2025-05-20.
- [33] R. Patel *et al.*, "Mosaic: A gpu dsl for modular ai acceleration," *arXiv preprint arXiv:2310.12345*, 2023.
- [34] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: Scaling compiler infrastructure for domain specific computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2021, pp. 2–14.
- [35] L. Adhianto, S. Banerjee, M. Fagan, M. Krentel, G. Marin, J. Mellor-Crummey, and N. R. Tallent, "Hpctoolkit: tools for performance analysis of optimized parallel programs <http://hpctoolkit.org>," *Concurr. Comput. : Pract. Exper.*, vol. 22, no. 6, pp. 685–701, Apr. 2010.
- [36] K. Zhou, X. Meng, R. Sai, and J. Mellor-Crummey, "Gpa: A gpu performance advisor based on instruction sampling," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 115–125.
- [37] K. Zhou, J. Anderson, X. Meng, and J. Mellor-Crummey, "Low overhead and context sensitive profiling of gpu-accelerated applications," in *Pro-*

ceedings of the 36th ACM International Conference on Supercomputing, 2022, pp. 1–13.

- [38] AMD, *Using rocprofv3*, 2025, accessed: 2025-03-02. [Online]. Available: <https://rocm.docs.amd.com/projects/rocprofiler-sdk/en/latest/how-to/using-rocprofv3.html>
- [39] V. Tsybal and A. Kurylev, “Profiling heterogeneous computing performance with vtune profiler,” in *Proceedings of the 9th International Workshop on OpenCL*, 2021, pp. 1–1.
- [40] N. Corporation, “Nvidia dlprof: Deep learning profiler,” <https://developer.nvidia.com/DLProf>, 2021, accessed: 2024-10-16.
- [41] JAX Team, “Jax profiler,” <https://jax.readthedocs.io/en/latest/profiler.html>, 2024, accessed: 2024-10-12.
- [42] PyTorch Team, *PyTorch Profiler*, 2024, accessed: 2025-05-08. [Online]. Available: <https://docs.pytorch.org/docs/stable/profiler.html>
- [43] Amazon Web Services, *Use Amazon SageMaker Profiler to Monitor and Debug Training Jobs*, 2024, accessed: 2025-05-08. [Online]. Available: <https://docs.aws.amazon.com/sagemaker/latest/dg/train-use-sagemaker-profiler.html>
- [44] J. Gleeson, S. Krishnan, M. Gabel, V. J. Reddi, E. de Lara, and G. Pekhimenko, “RI-scope: Cross-stack profiling for deep reinforcement learning workloads,” *arXiv preprint arXiv:2102.04285*, 2021. [Online]. Available: <https://arxiv.org/abs/2102.04285>
- [45] D. Snider, F. Charletier, and G. Pekhimenko, “Hotline profiler: Automatic annotation and a multi-scale timeline for visualizing time-use in dnn training,” in *Proceedings of the 2023 ACM Symposium on User Interface Software and Technology (UIST)*, 2023. [Online]. Available: <https://danielsnider.ca/papers/Hotline.pdf>
- [46] G. X. Yu, T. Grossman, and G. Pekhimenko, “Skyline: Interactive in-editor computational performance profiling for deep neural network training,” in *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2020. [Online]. Available: <https://arxiv.org/abs/2008.06798>
- [47] NVIDIA Corporation, “Cupti: Cuda profiling tools interface,” 2025, accessed: 2025-09-10. [Online]. Available: <https://docs.nvidia.com/cupti/>
- [48] AMD, “Roc-tracer: Rocm tracing library,” 2025, accessed: 2025-09-10. [Online]. Available: <https://rocm.docs.amd.com/projects/roctracer/>
- [49] I. Corporation, *oneAPI Level Zero Specification*, oneAPI Project, August 2025, accessed: 2025-09-11. [Online]. Available: <https://oneapi-src.github.io/level-zero-spec/>
- [50] “Perfetto: A production-grade open-source suite for system profiling, app tracing, and trace analysis,” <https://perfetto.dev/>, 2025, accessed: 2025-09-06.
- [51] A. Bhatele, S. Brink, and T. Gamblin, “Hatchet: pruning the overgrowth in parallel profiles,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: <https://doi.org/10.1145/3295500.3356219>
- [52] B. Gregg, “FlameGraph,” <https://github.com/brendangregg/FlameGraph>, n.d., accessed: 2025-09-06.
- [53] libunwind Project, *libunwind: a portable and efficient C API for unwinding stacks*, GNU Project / NONGNU, 2025, portable and efficient C programming interface to determine the call-chain of a program and manipulate call frames. [Online]. Available: <https://www.nongnu.org/libunwind/>
- [54] P. S. Foundation, *PyFrame_GetCode (Python C API)*, 2025, part of the Stable ABI since Python 3.10; returns a strong reference to the frame’s PyCodeObject. [Online]. Available: <https://docs.python.org/>
- [55] P. Team, “Deep dive on the hopper tma unit for fp8 gemms,” PyTorch Developer Blog, July 2024, explains advantages of Hopper’s TMA for FP8 tensor transfers.
- [56] P. Chen, M. Wahib, S. Takizawa, and S. Matsuoka, “Persistent kernels for iterative memory-bound gpu applications,” in *Proceedings of a relevant conference*, 2022, proposes a systolic-like persistent kernel model on CUDA GPUs.
- [57] P. Team, “Enabling advanced gpu features in pytorch – warp specialization,” PyTorch Developer Blog, February 2025, describes Triton 3.2’s warp specialization support on Hopper GPUs.
- [58] NVIDIA Corporation, *cuBLAS Library, Release 13.0*, Aug. 2025, aPI Reference Guide. [Online]. Available: https://docs.nvidia.com/cuda/pdf/CUBLAS_Library.pdf
- [59] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [60] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “Sglang: Efficient execution of structured language model programs,” *arXiv preprint arXiv:2312.07104*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [61] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” *arXiv preprint arXiv:2309.06180*, 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [62] NVIDIA Corporation, “NVTX: Nvidia tools extension library,” <https://nvidia.github.io/NVTX/>, 2025, accessed: 2025-08-31.
- [63] “vllm benchmarks: Datasets,” <https://docs.vllm.ai/en/latest/api/vllm/benchmarks/datasets.html>, 2025, accessed: 2025-09-10. [Online]. Available: <https://docs.vllm.ai/en/latest/api/vllm/benchmarks/datasets.html>
- [64] U. AI, D. Han-Chen, and M. Han-Chen, “Unsloth,” <https://github.com/unslothai/unsloth>, 2025.
- [65] A. Grattafiori, A. Dubey *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [66] Meta AI, “Introducing llama 3.1: Our most capable models to date,” <https://ai.meta.com/blog/meta-llama-3-1/>, 2024, accessed 2025-09-07.
- [67] —, “Llama 3.3 70B Instruct model card,” https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_3/, 2024, accessed 2025-09-07.
- [68] A. Kamath *et al.*, “Gemma 3 technical report,” *arXiv preprint arXiv:2503.19786*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.19786>
- [69] A. Q. Jiang *et al.*, “Mistral 7b,” *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [70] M. Abdin *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” *arXiv preprint arXiv:2404.14219*, 2024, describes the phi-3.5 series including phi-3.5-mini-instruct. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [71] L. Ben Allal *et al.*, “Smollm2: When smol goes big — data-centric training of a small language model,” *arXiv preprint arXiv:2502.02737*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.02737>
- [72] Hugging Face TB Research, “Smollm2-1.7b-instruct model card,” <https://huggingface.co/HuggingFaceTB/SmolLM2-1.7B-Instruct>, 2025, accessed 2025-09-07.
- [73] P. Zhang *et al.*, “Tinyllama: An open-source small language model,” *arXiv preprint arXiv:2401.02385*, 2024. [Online]. Available: <https://arxiv.org/abs/2401.02385>
- [74] L. Tunstall *et al.*, “Zephyr: Direct distillation of lm alignment,” *arXiv preprint arXiv:2310.16944*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.16944>
- [75] HuggingFace H4, “Zephyr-7b model card,” <https://huggingface.co/HuggingFaceH4/zephyr-7b-alpha>, 2023, accessed 2025-09-07.
- [76] A. Yang *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [77] S. Agarwal *et al.*, “gpt-oss-120b & gpt-oss-20b model card,” *arXiv preprint arXiv:2508.10925*, 2025. [Online]. Available: <https://arxiv.org/abs/2508.10925>
- [78] L. Atkins and D. MacLeod, “How to accurately time cuda kernels in pytorch,” Blog post on the Speechmatics website, Mar. 2023, accessed: YYYY-MM-DD. [Online]. Available: <https://www.speechmatics.com/company/articles-and-news/timing-operations-in-pytorch>
- [79] Textualize, “Textual — rapid application development framework for python,” <https://textual.textualize.io>, accessed: 2025-09-08.
- [80] K. Zhou, T. Zhong, H. Wu, J. Lee, G. Yue, Y. Ding, C. Robeck, J. Niu, and P. Tillet, “Proton: Towards multi-level, adaptive profiling for triton (artifact),” 2025, artifact available on Zenodo. DOI: 10.5281/zenodo.17972732. [Online]. Available: <https://doi.org/10.5281/zenodo.17972732>